Convolutional Networks, and Course Synthesis — Key Concepts

Lesson 10 — Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

4 December 2026 · a one-page index

Contents

Why a dense layer is the wrong shape 1
The operation 1
What makes it work 2
Teaching an invariance instead of asserting it 2
Which assumption is actually doing the work 2
Transfer learning 3
What the course adds up to 3

An index, not a summary. Each entry says what a term means and where the handout treats it properly. Use this to find your way back into the text, or to check before an exercise that there is no word here you could not define.

Why a dense layer is the wrong shape

The problem with one weight per pixel. A unit that has learned to spot a feature at one position knows nothing about the same feature three pixels away: those are different weights, trained by different examples. → § 2

Weight sharing. Using the same small set of weights at every position. What it buys is not memory but **evidence** — one example anywhere teaches the detector everywhere. → § 2

The operation

Kernel. The small array of weights that slides over the image. → § 3.1

Convolution. At each position, the weighted sum of the pixels the kernel covers. Strictly this is cross-correlation; since the kernel is learned, the distinction changes nothing that matters. → § 3.1

Feature map. The array of responses a single kernel produces over the whole image. → § 3.1

Padding and stride. How many pixels are added round the edge, and how far the kernel jumps. A three-by-three kernel with one pixel of padding leaves the size unchanged; a stride of two halves it. → § 3.2

Counting the output size. The last window can start at $n + 2p - f$; step by s from zero and count the starts, the $+1$ being the window at zero. → § 3.2

Kernel depth. Always equal to the input's channel count — only the two spatial dimensions are yours to choose, which is where parameter counts usually go wrong. → § 5

Zero-sum kernel. One whose weights add to zero, and which is therefore blind to absolute brightness and responds only to local contrast. → § 3.3

What makes it work

Equivariance to translation. Shift the input and the output shifts by the same amount: the response does not change, it moves. → § 4.1

Invariance. What pooling turns equivariance into: one number saying whether a strong response occurred anywhere, with the position discarded. → § 4.2

Max pooling. Take the maximum over a window. It converts equivariance into invariance, enlarges the receptive field, and discards spatial precision — a cost whenever *where* is part of the answer. → § 4.2

Receptive field. How much of the original image one unit deep in the network can see. Pooling grows it without adding weights. → § 4.2

Global pooling. The maximum over the whole feature map: one number per kernel, which is what makes the answer independent of position. → § 4.2

Inductive bias. An assumption built into the architecture rather than learned. A convolutional layer is a *restricted* dense layer — strictly less expressive — and the restriction is the entire value. → § 6

Why less expressive is better here. The functions it gives up are the ones that treat one row differently from another, and on this problem nobody wanted any of them. A model that cannot express a wrong answer does not have to learn to avoid it. → § 6

Teaching an invariance instead of asserting it

Data augmentation. Applying transformations the label is known to survive, so the model meets variations it would otherwise never see. → § 7

Augmentation against architecture. One is taught an example at a time, approximate, and can be forgotten; the other is asserted, exact, and free. Use augmentation for invariances no layer can express. → § 7

Two traps in comparing them. Give both arms the same number of *epochs*, not the same wall-clock time; and draw the transformation per image, not per batch, or you have added a few more fixed special cases. → § 7

Sample efficiency. How much data a model needs to reach its ceiling. The right inductive bias shows up here more starkly than in any single score. → § 8

Which assumption is actually doing the work

The permutation test. Shuffle the pixel positions consistently across every image, then retrain. Anything that relies on adjacency should suffer; anything that does not, will not notice. → § 9

Weight sharing against locality. Two separate assumptions bundled into one slogan. Deciding *whether* something is present needs only the first; deciding *what shape* it is needs the second. → § 9

Transfer learning

Transfer learning. Starting from the weights of a network trained on a related task, on the argument that early layers detect local structure and nothing about that is task-specific. → § 10

Why the source task decides everything. Transfer carries whatever the source *forced* the network to learn, and nothing else. Ask what the source actually required before assuming its features generalise. → § 10.1

Warm start against a frozen base. Freezing is a bet that the borrowed features are already right. When in doubt, warm start and leave everything trainable: that recovers from a bad source, and a frozen base cannot. → § 10.3

A window, not a slope. Too little data and transfer has nothing to attach to; too much and it has nothing to add. The gain is largest in between, not at the scarcest end. → § 10.2

Negative transfer. A badly chosen source leaving the model worse than starting from noise, because its features are committed to reporting the wrong thing. → § 10.3

What the course adds up to

Batch normalisation and residual connections. The two repairs that make deep stacks trainable — Lesson 9's vanishing gradient and the signal-size problem behind it, met again at depth. Named here, not covered, because these images do not need them. → § 11

Representations over models. A model turns a representation into a decision, and the representation usually matters more. Classical methods make you build it; convolutional networks learn it, paying in data and compute. Neither is a default. → § 12

Match the bias to the problem. A tree ensemble's own assumption — the answer depends on a few features crossing thresholds — can beat a network on image pixels, and a dense network can lose to logistic regression. → § 12.1